Informe TP3
Tecnología Digital 6

OFarrell, Rotondaro y Gonzalez Lobo

INTRODUCCIÓN

Este trabajo práctico se centró en la creación de un modelo para predecir géneros musicales, basándonos en un código base proporcionado por los profesores. A partir de esta base, exploramos y optimizamos distintos aspectos del modelo mediante experimentos en los que variamos hiperparámetros clave en cada punto y seleccionamos la mejor configuración para avanzar al siguiente punto, considerando tanto los mejores como los peores resultados obtenidos en cada caso (midiendo la Validation Loss y Accuracy de los modelos). Los experimentos incluyeron la configuración y monitoreo de la notebook en Weights and Biases para un registro detallado de los hiperparámetros, la exploración de distintas arquitecturas de red densa y CNN, la comparación de funciones de activación, optimizadores y *schedulers*, y el análisis de distintos batch-sizes y épocas de entrenamiento. También se evaluaron técnicas de regularización y se contrastaron los resultados utilizando espectrogramas frente a *waveforms* como entrada. Finalmente, se realizó una evaluación global con el set de test utilizando la configuración de mejor desempeño obtenida.

ARQUITECTURA

Para este experimento, se probó una arquitectura de red neuronal profunda variando la cantidad de capas ocultas, el tamaño de las capas densas y el número de nodos en cada capa oculta.

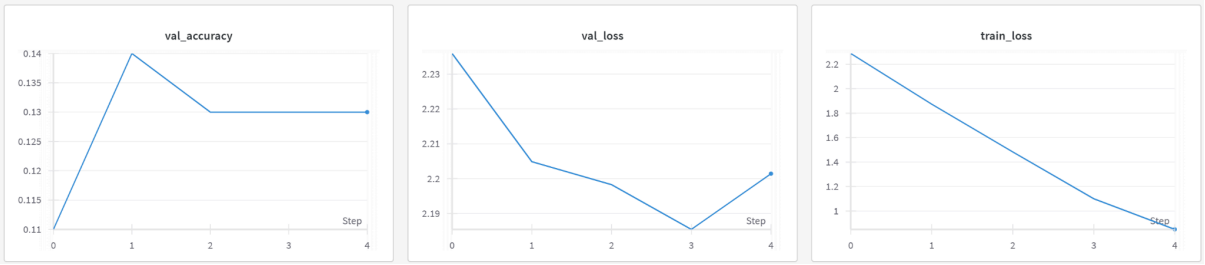
Nuestra estrategia fue partir de una arquitectura de una capa oculta y luego incrementar gradualmente la cantidad de capas y nodos para observar cómo afectaban al rendimiento. Inicialmente probamos con una sola capa oculta de 32, 64, y 128 nodos, y luego incrementamos a dos y tres capas con los mismos tamaños de nodos para cada capa.

Al observar los resultados, notamos que el aumento de nodos mejoraba el rendimiento hasta cierto punto, pero agregar demasiadas capas y nodos hacía que el modelo se sobreajustara rápidamente o no aprendiera adecuadamente. Con base en estos experimentos, determinamos que la arquitectura de dos capas con 64 nodos ofrecía la mejor combinación de capacidad de aprendizaje y generalización en el conjunto de validación.

Mejor Resultado:

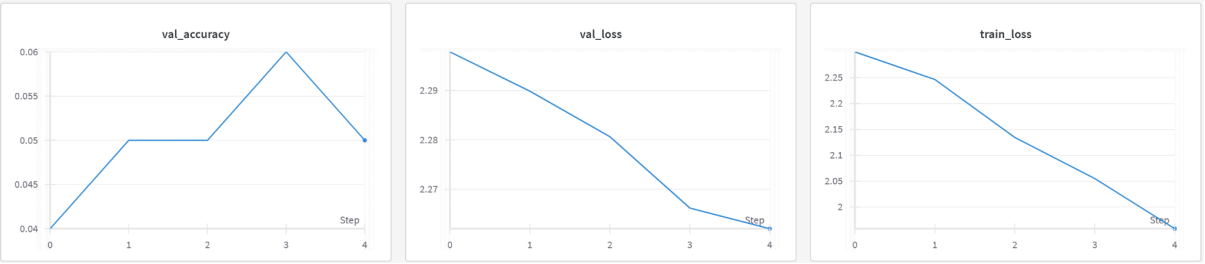
- Capas ocultas: 2
- Neuronas en cada capa oculta: 64

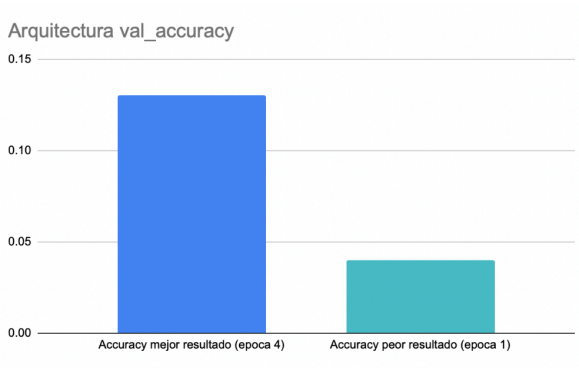
- Batch size: 20
- Epochs: 5

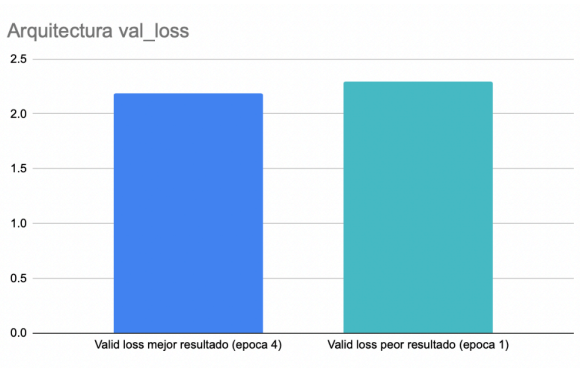


Peor resultado:

- Capas ocultas: 3
- Neuronas en cada capa oculta: 32
- Batch size: 20
- Epochs: 5







ARQUITECTURA CNN:

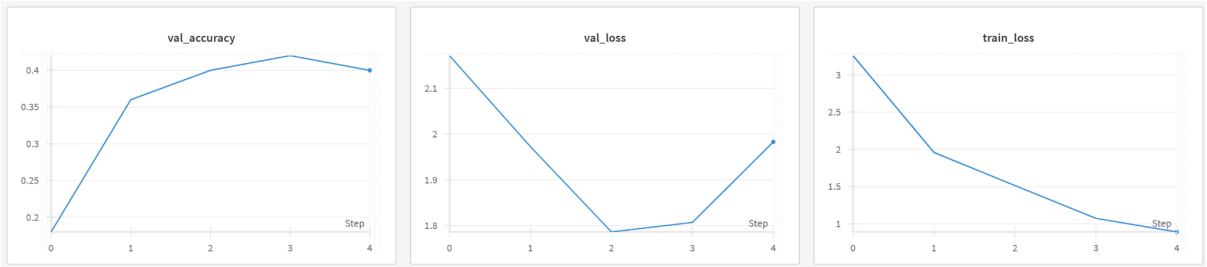
Para este experimento, extendimos nuestro análisis implementando arquitecturas de redes neuronales convolucionales (CNN) con diferentes cantidades de capas convolucionales. Nuestra estrategia fue comenzar con una arquitectura CNN básica de dos capas convolucionales y luego incrementar gradualmente la cantidad de capas hasta llegar a cuatro. Probamos cada configuración con un batch size de 20 y un total de 5 épocas de entrenamiento para observar cómo variaba el train_loss, valid_loss y la valid_accuracy.

Observamos que al incrementar el número de capas convolucionales, el modelo tendía a capturar mejor las características de los datos hasta cierto punto. La

configuración de cuatro capas convolucionales fue la que ofreció los mejores resultados, alcanzando una valid_accuracy máxima de 0.42 y un valid_loss de 1.7861 en la tercera época. Sin embargo, agregar más capas puede incrementar el riesgo de sobreajuste, por lo que ajustamos cuidadosamente el modelo para evitarlo. El peor resultado fue con 2 capas convolucionales ya que aunque mostró un Accuracy alto, el Valid Loss fue alto también.

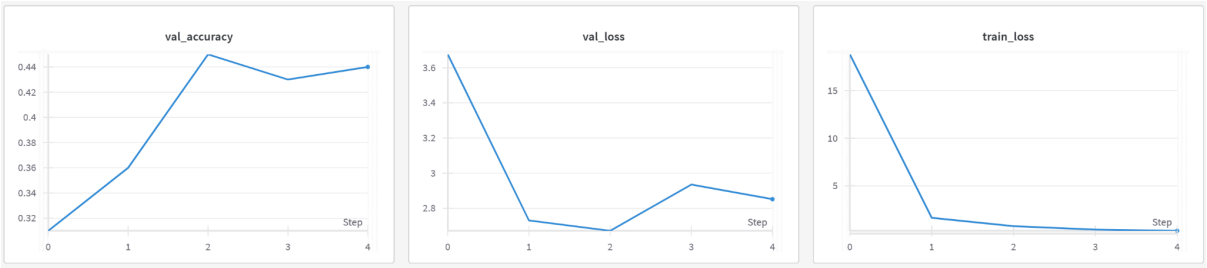
Mejor Resultado:

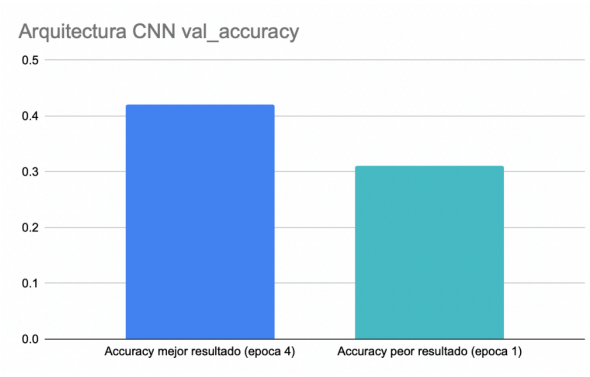
- Capas convolucionales: 4
- Batch size: 20
- Epochs: 5

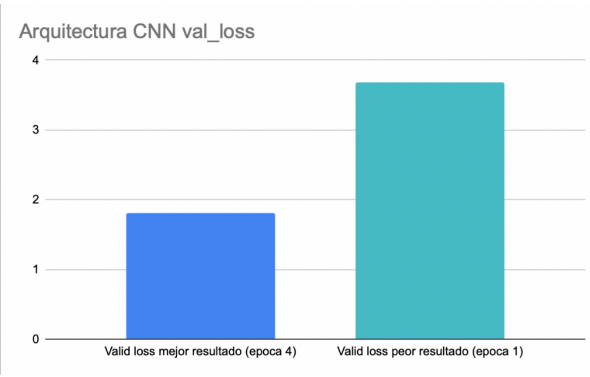


Peor resultado:

- Capas convolucionales: 2
- Batch size: 20
- Epochs: 5







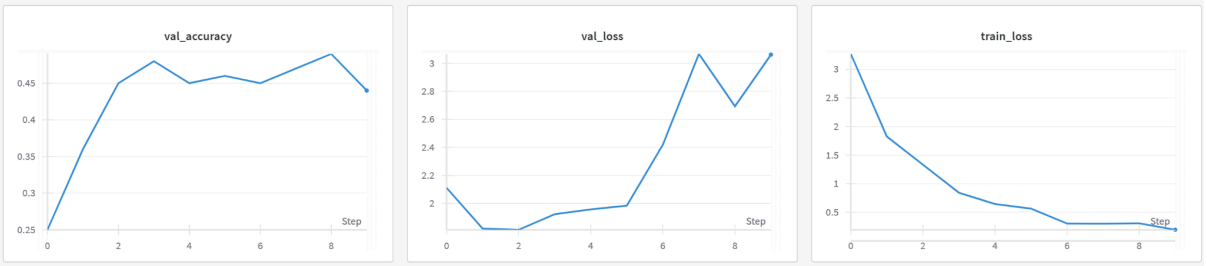
ENTRENAMIENTO

En este experimento, evaluamos el rendimiento del modelo variando los tamaños de batch y la cantidad de épocas de entrenamiento. Nuestra estrategia consistió en probar distintos valores para el batch size (5, 10, 15, 20, y 25) y el número de épocas (5 y 10), manteniendo fija la arquitectura del modelo con cuatro capas convolucionales. Observamos el impacto de estas configuraciones en la pérdida de entrenamiento (train_loss), la pérdida de validación (valid_loss) y la precisión en validación (valid_accuracy).

Al analizar los resultados, notamos que el mejor rendimiento se obtuvo con un batch size de 10 y un total de 10 épocas, logrando una valid_accuracy de 0.49 y un valid_loss de 2.6932. Aunque observamos un aumento en el valid_loss a medida que incrementamos las épocas, consideramos que 5 épocas eran insuficientes para un entrenamiento adecuado del modelo. Un batch size moderado combinado con un número suficiente de épocas permitió un aprendizaje más efectivo y una mejor generalización en el conjunto de validación.

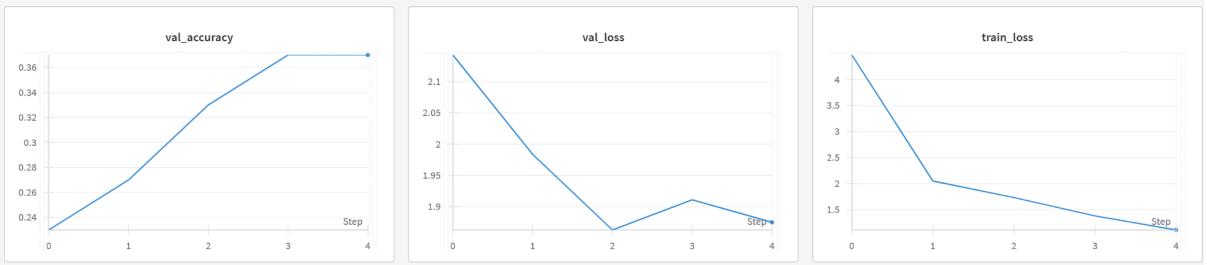
Mejor Resultado:

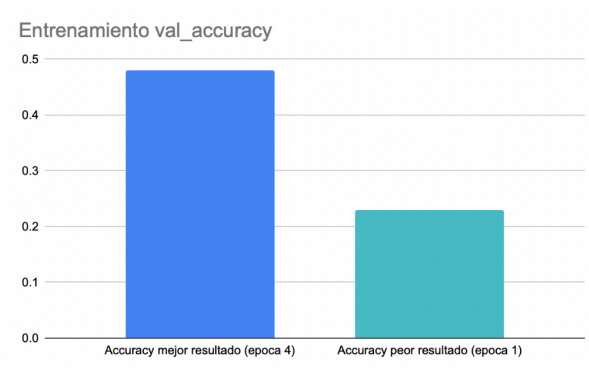
- Capas convolucionales: 4
- Batch size: 10
- Epochs: 10

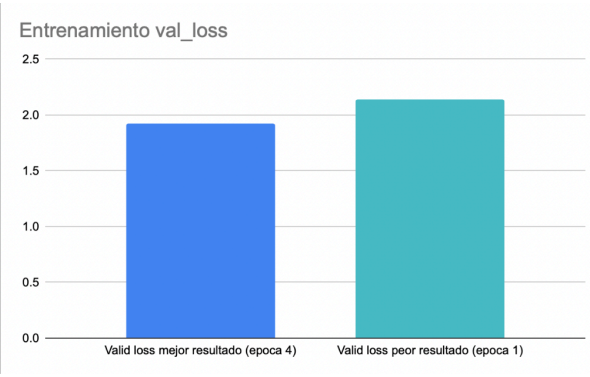


Peor Resultado:

- Capas convolucionales: 4
- Batch size: 25
- Epochs: 5







FUNCIONES DE ACTIVACIÓN

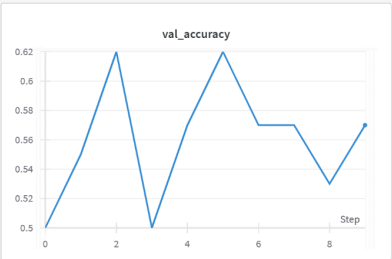
En este experimento, probamos diferentes funciones de activación en una arquitectura de CNN con cuatro capas convolucionales, batch size de 10 y 10 épocas, con el objetivo de determinar cuál de ellas maximizaría el rendimiento en validación.

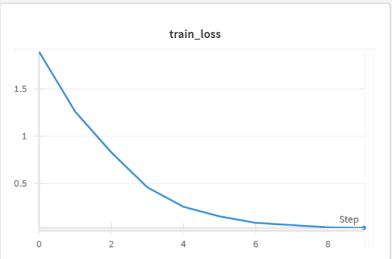
Probamos las siguientes funciones de activación: ReLU, Leaky ReLU, tanh, y sigmoide. La estrategia consistió en ejecutar cada modelo manteniendo las demás configuraciones constantes y observando la evolución de la pérdida en validación (valid_loss) y la precisión en validación (valid_accuracy). De este modo, pudimos identificar la función de activación que mejor equilibraba el aprendizaje del modelo y la generalización en el conjunto de validación.

Al analizar los resultados, observamos que la función tanh produjo el mejor desempeño, alcanzando una valid_accuracy de 0.62 y una valid_loss mínima de 1.2134. La función ReLU, aunque cercana en rendimiento, no superó a tanh. Leaky ReLU también mostró buenos resultados, pero no alcanzó la misma estabilidad en la validación. Por otro lado, la función sigmoide fue la de peor rendimiento, con una valid_accuracy que no superó el 8%, lo que indica una dificultad significativa para el modelo de aprender patrones en los datos con esta función de activación.

Mejor Resultado:

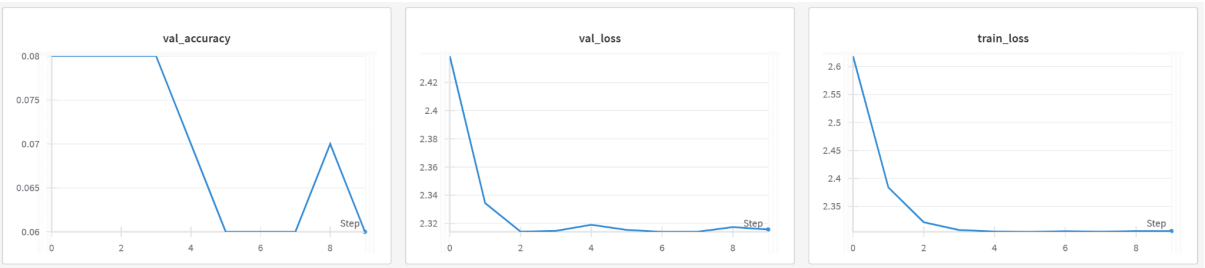
- Capas convolucionales: 4
- Batch size: 10
- Epochs: 10
- Función de activación: tanh

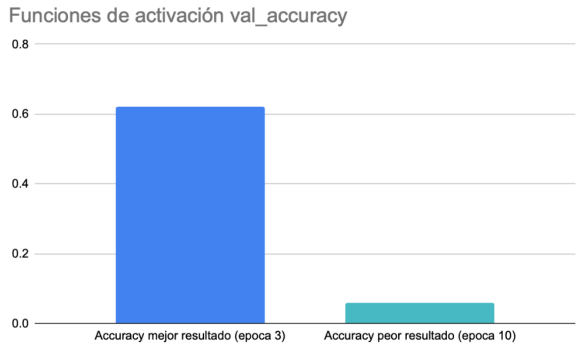


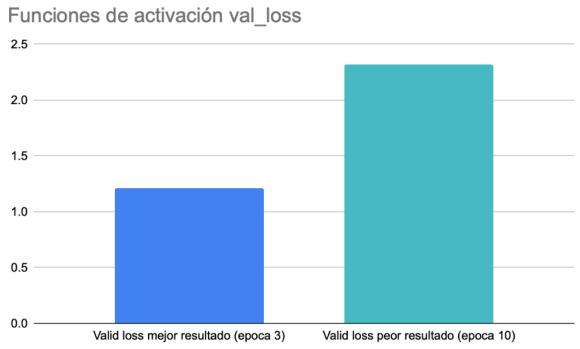


Peor Resultado:

- Capas convolucionales: 4
- Batch size: 10
- Epochs: 10
- Función de activación: sigmoid







OPTIMIZADORES

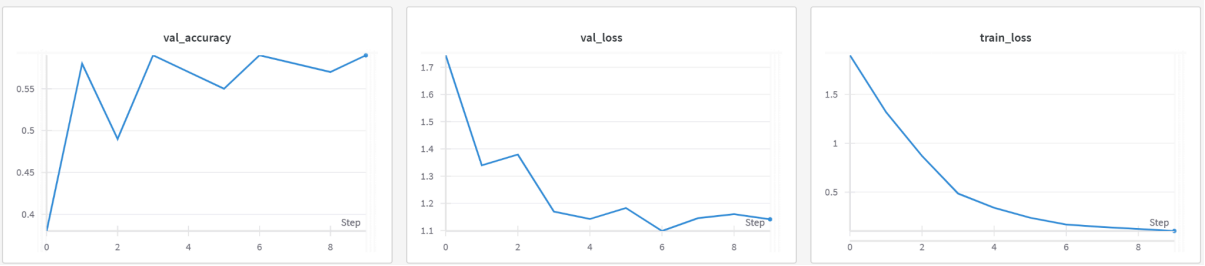
En este experimento, evaluamos el desempeño de diferentes optimizadores y schedulers en una arquitectura de CNN con cuatro capas convolucionales, batch size de 10, 10 épocas y la función de activación tanh.

Probamos los optimizadores Adam, SGD y RMSprop, y también aplicamos schedulers como ReduceLROnPlateau y StepLR sobre Adam, variando el learning rate de acuerdo con el progreso del entrenamiento. La estrategia consistió en mantener constantes todos los demás parámetros y observar cómo afectaba cada combinación de optimizador y scheduler a la evolución de valid_loss y valid_accuracy.

Al analizar los resultados, el optimizador Adam con StepLR obtuvo el mejor desempeño, alcanzando una valid_accuracy de 0.59 y una valid_loss mínima de 1.0985. Esta combinación ofreció una buena capacidad de ajuste al aprendizaje y un control óptimo de la tasa de aprendizaje, adaptándose a medida que el modelo convergía. Adam sin scheduler también mostró buenos resultados, aunque ligeramente inferiores. En contraste, el uso de RMSprop produjo el peor rendimiento, con una valid_accuracy que no superó el 12%, indicando problemas de ajuste con esta arquitectura.

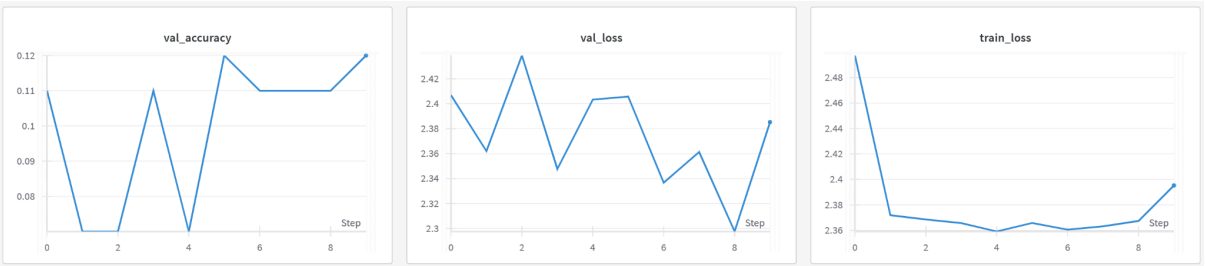
Mejor Resultado:

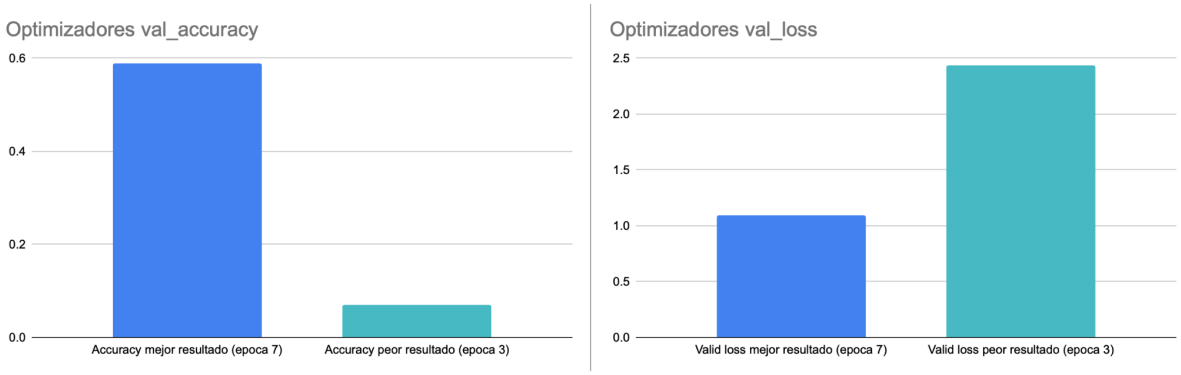
- Capas convolucionales: 4
- Batch size: 10
- Epochs: 10
- Función de activación: tanh
- Optimizador: Adam
- Scheduler: StepLR



Peor Resultado:

- Capas convolucionales: 4
- Batch size: 10
- Epochs: 10
- Función de activación: tanh
- Optimizador: RMSprop
- Scheduler: sin scheduler





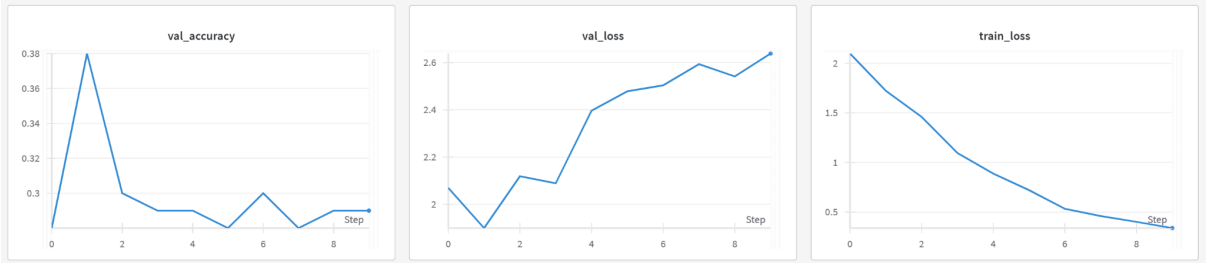
REGULARIZACIÓN

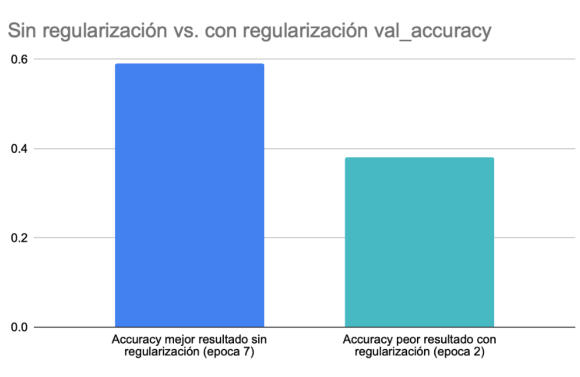
Se implementaron dos técnicas de regularización principales: **Dropout** con una probabilidad de 0.4, y **weight decay** en el optimizador Adam, utilizando además un scheduler StepLR para ajustar dinámicamente el learning rate. La estrategia fue mantener el resto de los parámetros constantes y observar cómo afectaba esta combinación a la evolución de valid_loss y valid_accuracy a lo largo de las épocas.

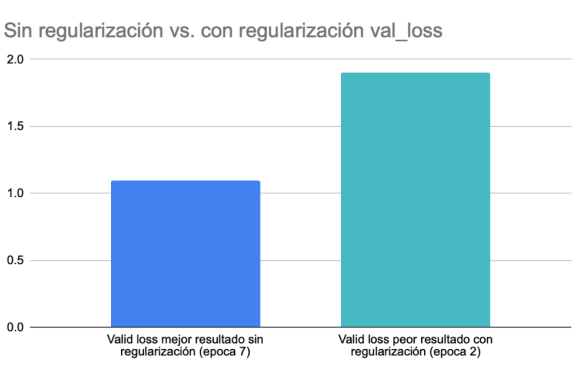
Al analizar los resultados, observamos que la regularización, en lugar de mejorar el rendimiento, condujo a una disminución en la accuracy y a un incremento en el val_loss. Aunque inicialmente se observó una mejora moderada en valid_accuracy al inicio del entrenamiento (con un máximo de 0.38 en la segunda época), la tendencia descendente en las siguientes épocas reflejó una pérdida en la capacidad de ajuste del modelo. Al final del entrenamiento, la valid_loss alcanzó valores altos (2.6387) y la valid_accuracy se estancó en 0.29, lo cual indica que la regularización aplicada afecto negativamente la capacidad de aprendizaje del modelo.

Hiperparámetros:

- Capas convolucionales: 4
- Batch size: 10
- Epochs: 10
- Función de activación: tanh
- Optimizador: Adam
- Scheduler: StepLR
- Regularizadores: dropout (0,4) y weigh decay







INPUT

Al principio intentamos usar los waveforms como input en el modelo, porque queríamos probar si funcionaban mejor en su formato de señal original. Pero nos topamos con problemas de compatibilidad en la CNN, ya que los waveforms tienen tamaños variables que complicaban el procesamiento. Por eso, decidimos transformarlos en espectrogramas, que muestran la frecuencia y el tiempo de la señal de audio en una imagen, y se adaptan mucho mejor a la estructura de la CNN. Para las redes densas al inicio sí pudimos usar waveforms directamente, pero al pasarnos a CNN tuvimos que cambiar a espectrogramas para aprovechar mejor este tipo de modelo.

EVALUACIÓN FINAL

Para la evaluación final, utilizamos un modelo CNN con cuatro capas convolucionales, el cual habíamos seleccionado en los experimentos previos y probamos en el conjunto de test. El modelo final tenía 10 epochs, un batch size de 10, función de activación tanh, optimizador Adam con un learning rate de 0.0001, y un scheduler StepLR con weight decay en el optimizador. En la última época (10/10), obtuvimos un test accuracy de 0.5400, lo que indica un desempeño moderado. Para validar la predicción a nivel individual, probamos con una muestra de audio del conjunto de test, pero, debido a la precisión alcanzada, la predicción no fue correcta. Esto refuerza que, aunque el modelo muestra potencial, aún tiene margen para mejorar en términos de precisión general.